



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #5
Sorting file data using array of pointers***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Sorting file data using array of pointers**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

When working with text files, sorting the content such as integers or words or line of strings can be efficiently managed using an **array of pointers**. Instead of moving entire content during sorting, we manipulate the pointers to those contents, which **reduces memory operations and improves performance**. This approach involves reading file contents into dynamically allocated memory blocks, storing addresses of these blocks in an array of pointers, and then applying a sorting algorithm (like selection sort) on the pointer array. This technique is especially useful for handling large text files with multiple lines or records, offering **both speed and flexibility in data processing**.

Let us consider a simple text file called numbers.txt which contains a lot of integers in a well structured manner. The requirement is to sort the integers in the file and write the sorted collection of integers to another file.

Coding Example_1:

```
#include <stdio.h>
#include <stdlib.h>
void swap( int** lhs, int** rhs );
void sort(int **aop, int n);
int main(){
    FILE* fp;
    fp = fopen("numbers.txt", "r");
    if(fp == NULL)
        printf("Can't open the file\n");
    else {
        int a;          int n = 0;
        while(fscanf(fp, "%d", &a) != -1)
            n++;
        int *p = malloc(n*sizeof(int));
        int **aop = (int **)malloc(n * sizeof(int *));
        for(int i = 0; i < n; i++)
            aop[i] = &p[i];
        rewind(fp);
        int i = 0;
        while(fscanf(fp, "%d", &p[i]) != -1)
            i++;
```

```
Displaying the contents of the dynamic array using array of integers
100    500    200    6    60    70    80    90
Displaying the contents of the dynamic array using array of pointers
s
100    500    200    6    60    70    80    90
After sorting
Displaying the contents of the dynamic array using array of integers
100    500    200    6    60    70    80    90
Displaying the contents of the dynamic array using array of pointers
s
6    60    70    80    90    100    200    500
Writing to a new file
Writing to a new file is done.
```

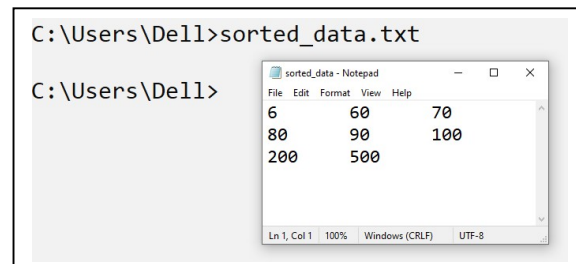
```

printf("\nDisplaying the contents of the dynamic array using array of integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", p[i]);
printf("\nDisplaying the contents of the dynamic array using array of pointers to
integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", *aop[i]);
sort(aop, n);
printf("\nAfter sorting\n");
printf("\nDisplaying the contents of the dynamic array using array of integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", p[i]);
printf("\nDisplaying the contents of the dynamic array using array of pointers to
integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", *aop[i]);
printf("\nWriting to a new file\n");
FILE *fpw = fopen("sorted_data.txt", "w");
for(int i = 0 ; i < n; i++)
    fprintf(fpw, "%d\t", *aop[i]);
printf("\nWriting to a new file is done.\n");
fclose(fp);
fclose(fpw);
}
return 0;
}

void sort(int **aop, int n){
    int i,pos,j;
    for(i = 0;i<n-1;i++)    {
        pos = i;
        for(j = i+1;j<n;j++)    {
            if(*aop[pos] > *aop[j])        pos = j;
        }
        if(pos != i)
            swap(&aop[pos],&aop[i]); // Swap the pointers not the integers in the array
    }
}

void swap( int** lhs, int** rhs){
    int* temp = *lhs;    *lhs = *rhs;    *rhs = temp;
}

```



Now let us consider the array of structures and array of pointers to array of structures. Given a data file **student_rollno_name.csv**, create a menu driven program to perform selection sort based on roll_number and name. Write the sorted record to a new file.

Data file Description:

- Contains the information of students in two columns only.
- First row represents the column headers such as roll_number and name.
- Contains around 55 rows. Every data is stored with a comma in between.

Coding Example_2:

```
struct student
{
    int roll;
    char name[20];
};

void init_ptr(struct student s[], struct student *p[], int n);
void swap( struct student** lhs, struct student** rhs);
void disp(struct student* p[], int n) ;
void selection_sort_roll_no(struct student* s[],int n);
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main(){
    FILE *fr=fopen("student_rollno_name.csv","r");
    char a[200]; fgets(a,200,fr);    char *item;
    struct student s[100]; int i=0;
    // Reading a line from the file and splitting it based on the delimiter comma(,),
    copying the details to array of student structure
    while(fgets(a,200,fr)) {
        item=strtok(a,",");
        s[i].roll=atoi(item);
        item=strtok(NULL,",");
        strcpy(s[i].name,item);
        i++;
    }
    int n = i;
    fclose(fr);
    struct student *p[100]; //Creation of array of pointers to point to array of structures
    init_ptr(s, p, n); // every element of array of pointer is made to point to every element of
    array of structure
    disp(p,n); // to display the array of structures using array of pointers
```

// Menu driven code

```
int ch;
printf("enter the choice.\n 1.sort on roll\n");
printf("2. sort on name");
scanf("%d",&ch);
switch(ch) {
    case 1:selection_sort_roll_no(p,n); // array of pointer and n is the argument
           disp(p,n); break;
    case 2:selection_sort_names(p,n); disp(p,n); break; // same as above
    default: printf("exiting from the program"); exit(0);
}
```

// Writing the sorted record to a new file

```
FILE *fw=fopen("student_sorted.csv","w");
fprintf(fw,"Roll_number,Name\n");
i=0;
while(i<n){
    fprintf(fw,"%d,%s",p[i]->roll,p[i]->name); i++;
}
fclose(fw);
printf("sorted data is written to a file student_sorted.csv\n");
return 0;
}
```

// Initializing the pointer

```
void init_ptr(struct student s[], struct student *p[], int n){
    for(int i = 0; i < n; ++i)
        p[i] = &s[i];
}
```

// Swapping the pointers

```
void swap( struct student** lhs, struct student** rhs) {
    struct student* temp = *lhs; *lhs = *rhs; *rhs = temp;
}
```

//Displaying using array of pointers

```
void disp(struct student* p[], int n) {
    for(int i = 0; i < n; ++i)
        printf("%d %s\n", p[i]->roll, p[i]->name);
    //printf("%s %d\n", (s+i)->name, (s+i)->roll);
}
```

//Selection sort using roll_no on array of pointers to array of structures

```
void selection_sort_roll_no(struct student *s[],int n){
    int i,pos,j;
    for(i = 0;i<n-1;i++)    {
        printf("in for\n");
        pos = i;
        for(j = i+1;j<n;j++)    {
            if((s[pos]->roll) > (s[j]->roll)) // compare the roll numbers using pointers to
sort the record
                pos = j;
        }
        if(pos != i)
            swap(&s[pos],&s[i]);    // Swap the pointers not the structures
    }
}
```

//Selection sort using name on array of pointers to array of structures

```
void selection_sort_names(struct student* s[],int n){
    int i,pos,j;
    for(i = 0;i<n-1;i++)    {
        pos = i;
        for(j = i+1;j<n;j++)    {
            if(strcmp(s[pos]->name,s[j]->name)>0)
                pos = j; // compare the names using pointers to sort the record
        }
        if(pos != i)
            swap(&s[pos],&s[i]);
    }
}
```

Happy Sorting file contents using Array of Pointers!!